

Redis: Novice to Platform Engineer/System Architect (7 YOY)

This handbook is built to move you from Redis basics to architecture-level decision making for high-scale production systems.

It mirrors the depth and structure of the Kafka chapter so your preparation stays consistent.

0) How to Use This Guide

Learning path:

- Level 1 (Beginner): sections 1 to 5
- Level 2 (Intermediate): sections 6 to 11
- Level 3 (Senior/Architect): sections 12 to 25

Interview pattern:

- Always explain trade-off: latency vs correctness vs durability vs cost.
-

1) Redis in One Minute

Redis is an in-memory data structure server used for caching, ephemeral state, real-time counters, queues, and low-latency coordination.

Core facts:

- Command execution is serialized per instance.
- Single command is atomic on one instance.
- Optional persistence exists (RDB, AOF).
- Replication is usually asynchronous.

Critical truth:

- Redis gives speed first, then configurable durability and availability.
-

2) Where Redis Fits (and Where It Does Not)

Best fits:

- Read-heavy caching
- Session/token storage
- Rate limiting and counters
- Leaderboards and ranking
- Short-lived distributed coordination

Not a default source of truth when:

- You need strict transactional guarantees
- You need strong consistency under failover
- You need deep audit/history semantics

Architect heuristic:

- Keep durable business truth in a database/log system; use Redis to accelerate access or coordination.
-

3) Core Internals and Execution Model

3.1 Event Loop and Atomicity

- Redis command path is single-threaded per instance.
- Background threads help with some I/O and maintenance.
- Long commands can block unrelated traffic and hurt tail latency.

3.2 Data Types and Encodings

- Types: string, hash, list, set, sorted set, stream, bitmap, hyperloglog, geospatial.
- Internal encodings can change with size/cardinality.
- This affects memory footprint and latency behavior.

3.3 Complexity vs Real Latency

- Big-O in docs is necessary but not enough.
 - Payload size, network, serialization, and key shape matter.
 - Avoid unbounded scans and giant values.
-

4) Persistence and Durability Model

RDB

- Point-in-time snapshots.
- Lower write overhead.
- Larger potential data-loss window between snapshots.

AOF

- Append write commands to log.
- Better durability window depending on fsync mode.
- More disk and rewrite overhead.

Typical production posture

- Use both, tuned by RPO/RTO and latency budget.

Durability caveat:

- Client-acknowledged writes can still be lost in some failover windows depending on replication/flush state.
-

5) Replication, Sentinel, and Cluster

Replication

- Primary-replica, asynchronous.
- Replica reads may be stale.

Sentinel

- Monitoring + automatic failover for non-sharded topology.

Redis Cluster

- Native sharding across hash slots (0 to 16383).
- Built-in failover per shard.
- Multi-key commands require same slot (or fail).

Key design concept:

- Hash tags like {user:42}:profile and {user:42}:prefs co-locate related keys.
-

6) Data Modeling for Production

String:

- Fast primitive for values and counters.

Hash:

- Field-level updates for object-like data.

List:

- Ordered queues/stacks with bounded usage patterns.

Set:

- Uniqueness and membership checks.

Sorted Set:

- Ranking, priority queues, time-ordered scheduling.

Stream:

- Append log with consumer groups.

Modeling rules:

- Keep values small.
 - Use predictable key naming.
 - Avoid unbounded growth without TTL/purge strategy.
-

7) Caching Patterns and Correctness

Cache-aside

- App reads cache first; on miss fetches DB then populates cache.

Write-through

- Write DB and cache synchronously.

Write-behind

- Cache accepts writes and flushes asynchronously.

Refresh-ahead

- Refresh hot keys before expiry.

Correctness caveat:

- Cache is a derived state; define stale-read tolerance per endpoint.
-

8) Invalidation, Stampede, and Hot Key Controls

Invalidation patterns:

- TTL-only
- Explicit delete on write
- Versioned keys

Stampede controls:

- TTL jitter
- Request coalescing/single-flight
- Stale-while-revalidate
- Per-key mutex with strict timeout

Hot key mitigation:

- Local in-process cache layer
 - Read replicas where safe
 - Key sharding for read-heavy counters
 - Precompute and fan-out updates
-

9) Transactions, Lua, and Distributed Locks

Atomic tools:

- Single command atomicity
- MULTI/EXEC batching
- Lua/EVAL for complex read-modify-write atomics

Distributed lock baseline:

- SET lock:key token NX PX ttl
- Release by compare-and-delete script only

Lock caveat:

- Locks are coordination primitives, not a replacement for durable transactional integrity.
-

10) Streams and Queueing with Redis

Streams capabilities:

- Append events with IDs
- Consumer groups
- Pending entries list
- Claim/reclaim for stuck consumers

When Streams are great:

- Lightweight async workflows
- Moderate retention and throughput needs

When to choose Kafka instead:

- Large replay windows
 - Massive fan-out and partitioned throughput
 - Stronger ecosystem for event backbone governance
-

11) Security and Governance

Security controls:

- TLS in transit
- ACL users/roles
- Authentication with strong secret handling
- Command/category restrictions per principal

Governance practices:

- Separate users for app classes
 - Rotation policy for credentials
 - Key prefix ownership by service/team
 - Audit access in managed environments where available
-

12) Capacity Planning (Senior)

Plan along dimensions:

- QPS and p95/p99 latency
- Dataset size and growth
- TTL distribution
- Replication factor
- Persistence overhead

Memory sizing rule:

- provisioned memory must include data + overhead + fragmentation + headroom.

Practical headroom:

- Keep operational margin to absorb traffic spikes and failover reshuffling.
-

13) Eviction and Memory Engineering

Common policies:

- noeviction
- allkeys-lru
- allkeys-lfu
- volatile-lru
- volatile-lfu

Policy selection principle:

- Match policy to business criticality and key lifecycle.

Fragmentation management:

- Track memory fragmentation ratio.
 - Avoid huge churn patterns.
 - Plan maintenance windows for heavy rewrite scenarios when needed.
-

14) SRE Observability and SLOs

Must-watch metrics:

- used_memory and headroom
- hit ratio and miss ratio
- evictions and expirations
- command latency percentiles
- blocked clients
- replication offset lag
- failover events

SLO examples:

- p99 GET latency under target
- cache hit ratio above target for critical paths
- zero unplanned failovers over rolling window target (or strict budget)

Alerting patterns:

- memory > threshold with positive growth slope
 - sudden miss-ratio jump post-deploy
 - replication lag above threshold for sustained period
-

15) Failure Modes and Runbooks

Failure classes:

- Hot key overload
- Eviction storm
- Persistence-induced latency spike
- Split-brain style failover confusion
- Network partition between primary and replicas

Runbook essentials:

- clear detection signal
 - immediate mitigation action
 - rollback criteria
 - data correctness verification checklist
-

16) Multi-AZ, Multi-Region, and DR

Design options:

- single-region with multi-AZ replicas
- active-passive regional DR
- active-active by domain partitioning (careful conflict model)

Architect questions:

- What is acceptable data loss window?
- How long can writes be unavailable?
- Can stale reads be tolerated during failover?

DR reality:

- Redis replication is asynchronous; define failover data-loss expectations explicitly.
-

17) Platform Engineering with Redis

Treat Redis as a shared platform product.

Platform capabilities:

- self-service instance/database provisioning
- standard key naming and TTL policy
- quota and tenant isolation model
- approved patterns (cache-aside, rate limit, lock, stream)
- reusable client wrappers with safe defaults

Governance KPI ideas:

- % keys with valid TTL policy
 - incident rate by misuse category
 - top 10 memory owners trend
-

18) Cost Engineering

Primary cost drivers:

- total memory footprint
- replica count
- persistence and I/O class
- cross-AZ/region traffic
- overprovisioned headroom

Optimization levers:

- shrink value payloads
- use hashes for sparse object updates when suitable
- enforce TTL hygiene
- tier non-critical caches to cheaper infrastructure
- remove low-value cache keys with poor hit rate

Anti-pattern:

- scaling Redis endlessly to hide poor database/query design.
-

19) Redis in Go: Production Patterns

Recommended client behavior:

- strict timeouts
- bounded retries with jitter

- circuit breaker around Redis dependency
- clear fallback behavior per endpoint

Simple fixed-window limiter in Go:

```
key := fmt.Sprintf("r1:%s:%d", userID, time.Now().Unix()/60)
n, err := rdb.Incr(ctx, key).Result()
if err != nil {
    return err
}
if n == 1 {
    _ = rdb.Expire(ctx, key, time.Minute).Err()
}
if n > 100 {
    return ErrRateLimited
}
return nil
```

Atomic unlock script:

```
if redis.call('GET', KEYS[1]) == ARGV[1] then
    return redis.call('DEL', KEYS[1])
end
return 0
```

20) Common Anti-Patterns

- Using KEYS in hot production paths.
 - Storing huge blobs and expecting microsecond latency.
 - No TTL discipline for cache-only keys.
 - Treating async replicas as strongly consistent reads.
 - Using distributed lock as sole correctness mechanism.
 - Ignoring hit ratio economics and cache value contribution.
-

21) Interview Question Bank (Beginner to Architect)

Beginner:

1. Why is Redis fast?
2. Difference between string, hash, set, sorted set?
3. What is TTL and why use jitter?

Intermediate:

1. RDB vs AOF trade-offs?
2. Sentinel vs Cluster?
3. How to handle cache stampede and hot keys?

Senior:

1. Design cache invalidation for correctness-sensitive pricing data.
2. How do you choose eviction policy per workload class?

3. How do you set SLOs and alerts for Redis-backed APIs?

Architect:

1. Design Redis topology for multi-region low-latency reads with controlled risk.
 2. How do you build governance for 100+ services sharing Redis?
 3. When should you replace Redis in a path instead of scaling it?
-

22) 90-Day Redis Mastery Roadmap

Days 1 to 15:

- Core data structures, TTL, and cache-aside.

Days 16 to 35:

- Persistence, replication, Sentinel/Cluster labs.

Days 36 to 55:

- Build rate limiter, lock utility, and stampede control.

Days 56 to 75:

- Benchmark latency under memory pressure and failover drills.

Days 76 to 90:

- Design platform standards and DR strategy; run game-day scenarios.
-

23) Practical Checklists

Workload checklist:

- read/write ratio known
- stale-read tolerance defined
- fallback path documented

Key design checklist:

- namespace/prefix standard
- TTL policy by key class
- size guardrails and cardinality controls

Reliability checklist:

- failover tested
- lag and latency alerts configured
- eviction behavior validated under pressure

Security checklist:

- ACL per service
 - TLS enforced where required
 - credential rotation process
-

24) High-Value Incident Stories to Prepare

Story 1:

- stampede at TTL boundary, mitigation with jitter + single-flight.

Story 2:

- hot key caused p99 spike, fixed via local cache + key redesign.

Story 3:

- failover data gap exposed durability assumption; corrected persistence and replication policy.

Story 4:

- cost reduction via TTL cleanup and low-value key retirement.
-

25) Final Architect Summary

A 7+ YOE platform engineer or architect using Redis well should be able to:

- Explain internals clearly (event loop, data structures, persistence, replication).
- Map business requirements to eviction, durability, and consistency settings.
- Design for failure and degraded mode, not only happy path latency.
- Build governance and standards for multi-team Redis usage.
- Optimize both reliability and cost with measurable SLOs.

If you can justify Redis decisions with explicit correctness and failure trade-offs, you are operating at architect level.